

Министерство цифрового развития, связи и массовых коммуникаций РФ  
Федеральное государственное бюджетное образовательное учреждение высшего  
образования

«Сибирский государственный университет  
телекоммуникаций и информатики» (СибГУТИ)

Кафедра телекоммуникационных систем и вычислительных средств

Расчетно-графическая работа

Реализация приема и передачи битовой последовательности в условиях помех

Выполнил: студент 3 курса

гр. ИА-331

Помелова А.В.

Новосибирск 2025

## ЦЕЛЬ РАБОТЫ

Закрепить и структурировать знания, полученные в рамках изучения дисциплины «Основы систем мобильной связи».

## ЗАДАНИЕ И ПОРЯДОК ВЫПОЛНЕНИЯ РАСЧЕТНО-ГРАФИЧЕСКОЙ РАБОТЫ

- 1) Введите с клавиатуры ваши имя и фамилию латиницей.
- 2) Сформируйте битовую последовательность, состоящую из  $L$  битов, кодирующих ваши имя и фамилию латинице ASCII-символов. Результат: массив нулей и единиц с данными и разработанный ASCII-кодер. Визуализируйте последовательность на графике.
- 3) Вычислите CRC длиной  $M$  бит для данной последовательности, используя входные данные для своего варианта из работы №5 и добавьте к битовой последовательности. Результат: CRC-генератор и выведенный в терминал CRC.
- 4) Для того, чтобы приемник смог корректно принимать такой сигнал и находить моменты начала, нужно реализовать синхронизацию. Для этого перед отправкой полученной последовательности добавьте последовательность Голда, которую вы реализовывали в работе №4, длиной  $G$ -бит. Результат: функция генерации последовательности Голда и массив с битами данных, CRC и синхронизации. Визуализируйте последовательность на графике.
- 5) Преобразуйте биты с данными во временные отсчеты сигналов, так чтобы на каждый бит приходилось  $N$ -отсчетов. Результат: массив длиной  $N \times (L+M+G)$  нулей и единиц – но это уже временные отсчеты сигнала (пример амплитудной модуляции). Визуализируйте последовательность на графике.
- 6) Создайте нулевой массив длиной  $2 \times N \times (L+M+G)$ . Введите с клавиатуры число от 0 до  $N \times (L+M+G)$  и в соответствие с введенным значением вставьте в него массив значений из п.5. Результат – массив `Signal` – визуализируйте на графике.
- 7) Предположим, что сформированная выше последовательность, промодулировала высокочастотное несущее колебание, передалась через радиоканал и на приемной стороне была оцифрована с заданной частотой дискретизации  $f_s$  (число отсчетов сигнала в 1 секунде). Проходя через канал отсчеты сигнала исказились (опустим пока историю с затуханием и изменением амплитуды) – к ним добавились значения шумов, присутствовавших в канале, которые можно получить, используя нормальный закон распределения с  $\mu=0$  и  $\sigma$  – вводится с клавиатуры (float). То есть нужно сформировать массив с шумом размером  $2 \times N \times (L+M+G)$ , реализовав его с помощью нормального распределения, например,

```
noise( $\sigma$ ) := rnorm(length(Signal),  $\mu$ ,  $\sigma$ )
```

Затем нужно поэлементно сложить информационный сигнал с полученным шумом. Визуализировать массив отсчетов зашумленного принятого сигнала.

8) Реализуйте функцию корреляционного приема и определите, начиная с какого отсчета (семпла) начинается синхросигнал в полученном массиве, удалите лишние биты до этого массива, выведите значение в терминал. Результат: функция корреляционного приемника.

9) Зная длительность в отсчетах  $N$  каждого символа, разберите оставшиеся символы. Накапливайте по  $N$  отсчетов и сравнивайте их с пороговым значением  $P$  (подумайте, какое значение порога следует выбрать, чтобы интерпретировать полученные семплы нулями или единицами). Напишите функцию, которая будет принимать решение по каждому  $N$  отсчетам – 0 передавался или 1, на выходе которой должно быть  $(L+M+G)$  битов данных. Лишние отсчеты можно отбросить.

10) Удалите из полученного массива  $G$ -бит последовательности синхронизации.

11) Проверьте корректность приема бит, посчитав CRC. Выведите в терминал информацию о факте наличия или отсутствия ошибки.

12) Если ошибок в данных нет, то удалит биты CRC и оставшиеся данные подайте на ASCII-декодер, чтобы восстановить посимвольно текст. Выведите результат на экран.

13) Визуализируйте спектр передаваемого и принимаемого (зашумленного) сигналов. Измените длительность символа, уменьшите ее в два раза и увеличьте тоже вдвое. Выведите на одном графике спектры всех трех сигналов (с короткими, средними и длинными символами).

14) Сделайте промежуточные выводы по каждому пункту работы и общее заключение.

15) Оформите работу. Отчет должен содержать титульный лист, содержание, цель и задачи работы, теоретические сведения, исходные данные, этапы выполнения работы, сопровождаемые скриншотами и графиками, демонстрирующими успешность выполнения, и промежуточными выводами, результирующими таблицами и заключение, и **ссылка в виде QR-кода на репозиторий с кодом (git)**.

## ВЫПОЛНЕНИЕ РАСЧЕТНО-ГРАФИЧЕСКОЙ РАБОТЫ

Мы моделируем цифровую систему связи:

1. Текст в биты
2. [данные + CRC]
3. [Gold][Данные + CRC]
4. Битовая последовательность в дискретный временной сигнал
5. Сигнал + шум
6. Прием и поиск начала пакета

7. Принятие решений (демодуляция)
8. Проверка CRC
9. Биты в текст

## 1. Ввод данных и параметров системы

```
name = input('Введите имя и фамилию: ', 's');  
  
N = 10;  
M = 7;  
G_1 = 31; % 2^5 - 1
```

С клавиатуры вводится строка, содержащая имя и фамилию латиницей.

Задаются параметры системы:

- N - количество временных отсчётов на один бит,
- M - длина CRC,
- G\_1 - длина синхропоследовательности Голда.

```
>> code  
Введите имя и фамилию: NastyaPomelova
```

Вывод: введены исходные данные и параметры системы, определяющие структуру сигнала и условия его передачи. Заданные параметры используются на всех последующих этапах моделирования.

## 2. ASCII-кодирование текстовой информации

- Каждый символ строки преобразуется в его ASCII-код (целое число от 0 до 255).
- ASCII-код каждого символа переводится в двоичную форму длиной 8 бит.
- Все полученные битовые последовательности объединяются в один массив.
- Полученная битовая последовательность визуализируется на графике.

dec2bin() - двоичное преобразование

- '0' - преобразует символьный массив в числовой

Поскольку в ASCII:

- '0' = 48

- '1' = 49

То:

- '0' - '0' = 48 - 48 = 0
- '1' - '0' = 49 - 48 = 1

Результирующая битовая последовательность имеет длину  $\text{длина\_строки} * 8$  бит

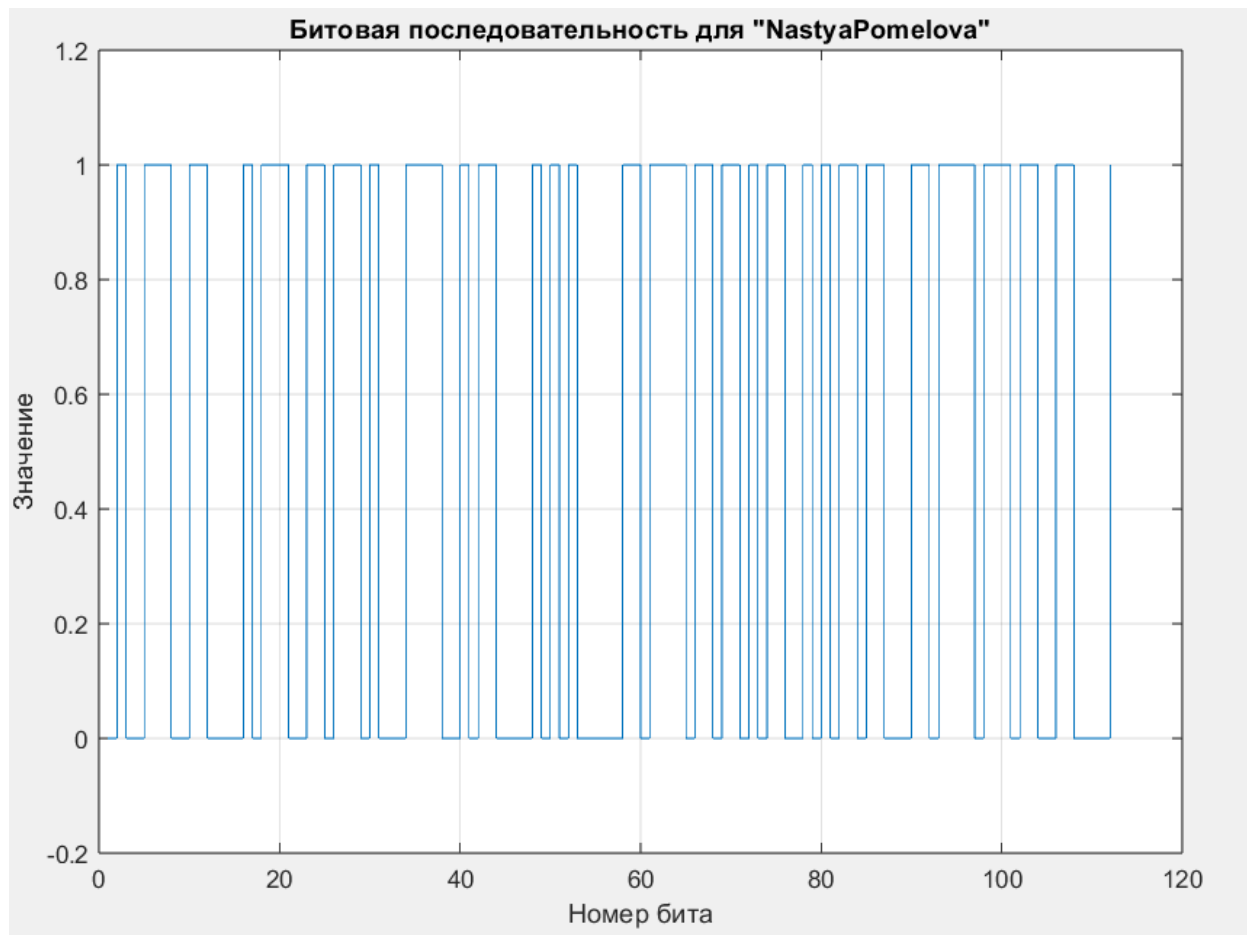
```
ascii = double(name);
l = length(ascii)*8;  %код 1 символа = 8 бит
```

```
bits = [];
for i = 1:length(ascii)
    b = dec2bin(ascii(i), 8) - '0';
    bits = [bits b];
end
```

```
fprintf('Битовая последовательность: ');
fprintf('%d', bits);
fprintf('\n');
```

```
figure;
stairs(bits);
ylim([-0.2 1.2]);
title(['Битовая последовательность для "', name, '"']);
xlabel('Номер бита');
ylabel('Значение');
grid on;
```

```
>> code
Введите имя и фамилию: NastyaPomelova
Битовая последовательность: 0100111001100001011100110111010001111001011000010101000001101111011011010110010101101100011011110111011001100001
```



Вывод: исходный текст успешно преобразован в двоичную последовательность на основе ASCII-кодирования, что позволяет представить символьную информацию в виде цифровых данных.

### 3. Контроль циклическим избыточным кодом (CRC)

CRC (Cyclic Redundancy Check) - метод обнаружения ошибок при передаче данных. Он позволяет обнаруживать одиночные, двойные и многие групповые ошибки.

Параметры CRC:

- Длина контрольной суммы = 7 бит
- Порождающий полином -  $G = x^7 + x^6 + x + 1$

Алгоритм вычисления CRC:

- К исходной битовой последовательности добавляются нулевые биты, равные длине CRC.
- Выполняется деление по модулю 2 на порождающий полином.
- Остаток от деления формирует контрольную сумму CRC.
- CRC добавляется к исходной битовой последовательности.

- Контрольная сумма выводится в терминал.

```

crc_l = M;
G = [1 1 0 0 0 1 1];          %x^7 + x^6 + x + 1

crc_data = [bits zeros(1,crc_l)];
temp = crc_data;

for i = 1:length(bits)
    if temp(i) == 1
        % Проверяем, что не выходим за границы
        if i <= length(temp) - crc_l
            temp(i:i+crc_l) = xor(temp(i:i+crc_l), G);
        end
    end
end

crc = temp(end - crc_l+1:end);

fprintf('CRC: ');
fprintf('%d', crc);
fprintf('\n');

bits_crc = [bits crc];

```

```
CRC: 1111001
```

Вывод: для битовой последовательности вычислена контрольная сумма CRC, обеспечивающая обнаружение ошибок при передаче данных по зашумлённому каналу.

#### 4. Генерация синхронизирующей последовательности Голда

Для корректного приёма пакета данных приёмнику необходимо определить момент начала сообщения. Для этого используются синхропоследовательности с хорошими корреляционными свойствами - например последовательность Голда.

Параметры:

- $B = 5$  - длина регистров сдвига
- $G_l = 31$  - длина последовательности Голда ( $2^5 - 1$ )
- Начальные состояния регистров вычисляются на основе параметра  $N$

Алгоритм:

- Берутся два регистра сдвига (простых счётчика на основе битов), в которых биты циклически сдвигаются вправо.

- В каждом регистре определённые биты складываются с помощью XOR и подаются обратно на вход - создается новая последовательность.
- Выходные биты регистров объединяются операцией XOR.
- Полученная последовательность Голда добавляется в начало пакета данных.
- Итоговая последовательность визуализируется на графике.

```
function gold = generate_gold(x, y, length)
    gold = zeros(1, length);
    x = x(:)';      % убедимся, что вектор-строка
    y = y(:)';

    for i = 1:length
        gold(i) = xor(x(5), y(5));      % выход = x5 XOR y5

        new_x1 = xor(x(4), x(5));      % x1 = x4 XOR x5
        new_y1 = xor(y(2), y(5));      % y1 = y2 XOR y5

        x = [new_x1, x(1:4)];
        y = [new_y1, y(1:4)];
    end
end

B = 5;
G_1 = 31; % 2^5 - 1

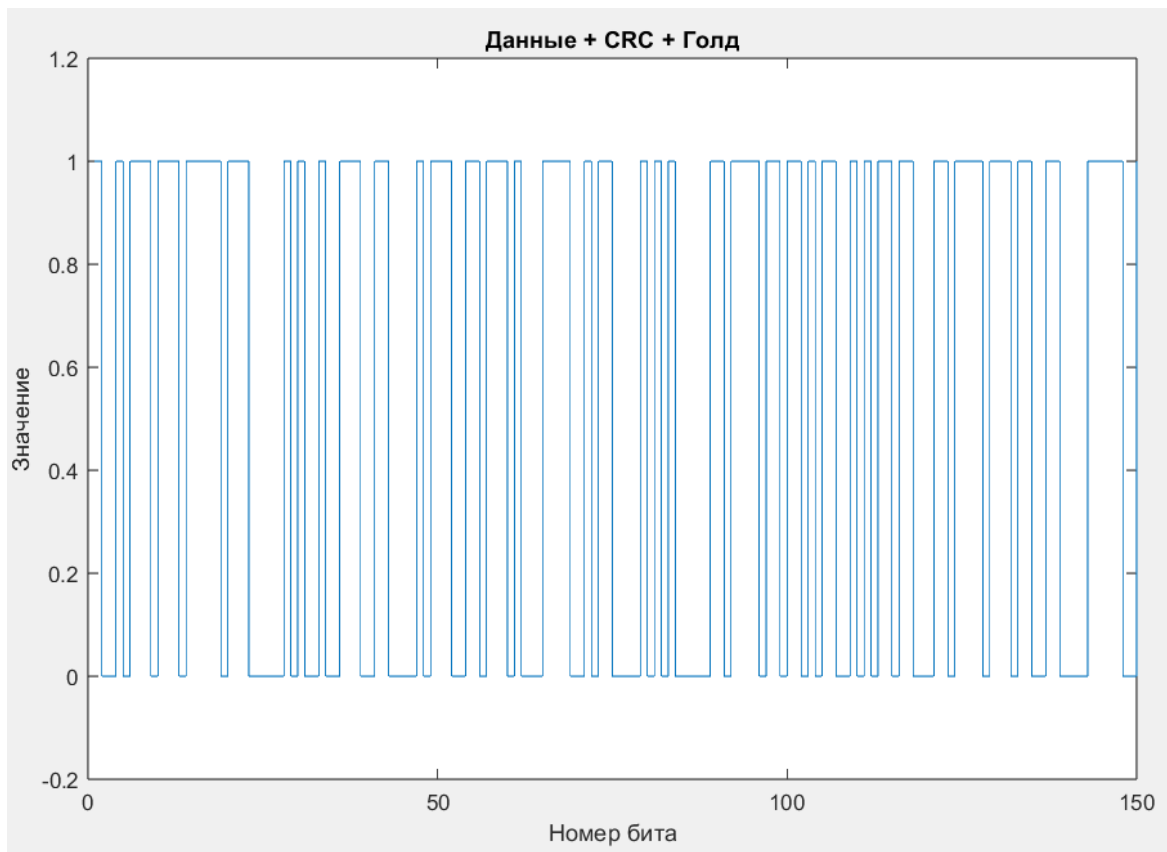
x = dec2bin(N, B)- '0';
y = dec2bin(N + 7, B)- '0';

gold = generate_gold(x, y, G_1);

tx_bits = [gold bits_crc];

figure;
stairs(tx_bits);
title('Данные + CRC + Голд');
ylim([-0.2 1.2]);
xlabel('Номер бита');
ylabel('Значение');
```





Вывод: сформирована последовательность Голда, позволяющая надежно определить начало передаваемого пакета на приёмной стороне.

## 5. Преобразование битов данных во временные отчеты сигнала

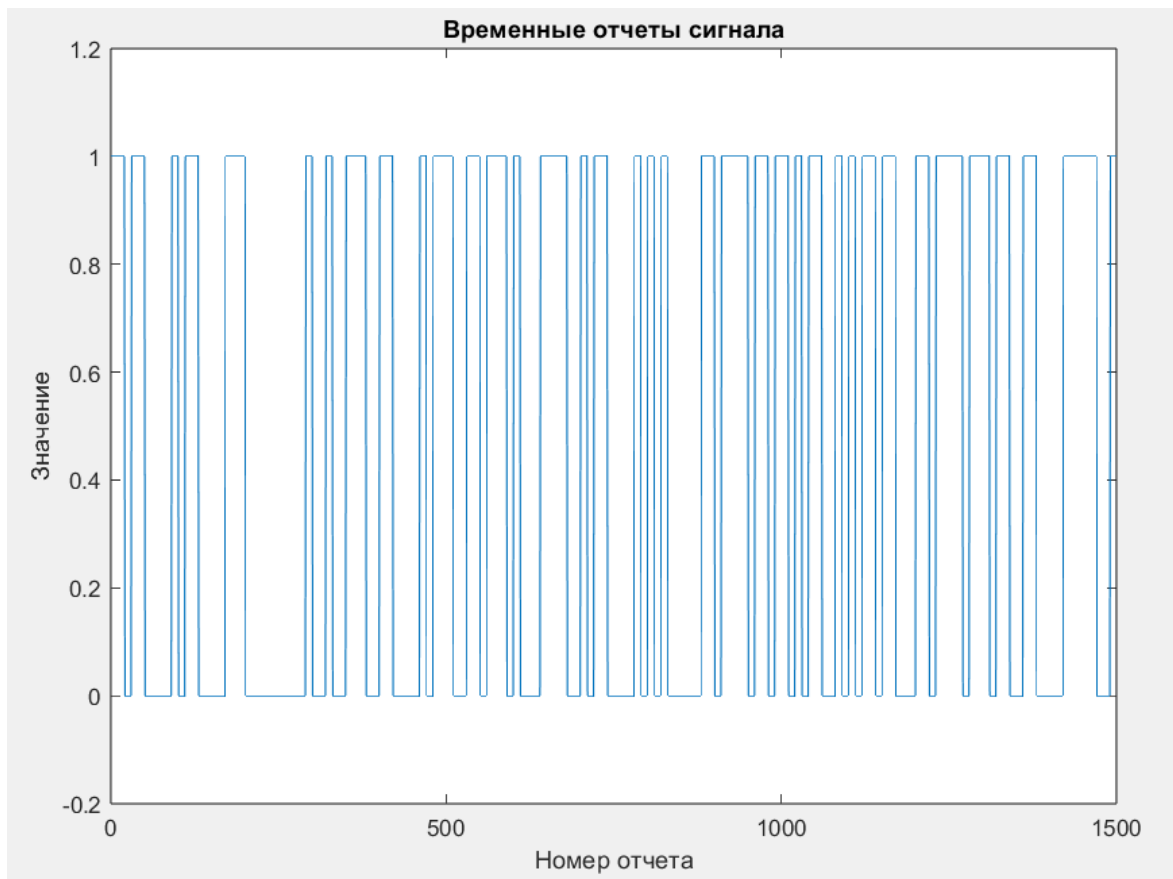
В реальных системах связи биты передаются не мгновенно, а в течение некоторого времени. Для моделирования этого процесса каждый бит заменяется набором одинаковых временных отсчётов.

Алгоритм:

1. Каждый бит повторяется N раз.
2. В результате формируется массив временных отсчётов в N раз больше

```
N = 10; %количество отчетов на бит
signal = repelem(tx_bits, N);
```

```
figure;
plot(signal);
title('Временные отчеты сигнала');
ylim([-0.2 1.2]);
xlabel('Номер бита');
ylabel('Значение');
```



Вывод: битовая последовательность преобразована во временной сигнал путём увеличения числа отсчётов на бит, что соответствует простой амплитудной модуляции.

## 6. Вставка сигнала в нулевой массив

В реальном канале передача сообщения может начинаться в произвольный момент времени. Для моделирования этого эффекта сигнал вставляется в более длинный массив, заполненный нулями.

Алгоритм:

1. Создаётся нулевой массив длиной в два раза больше сигнала.
2. Пользователь вводит величину сдвига.
3. Сигнал вставляется в массив, начиная с заданной позиции.

```
signal_l = length(signal);
new_signal = zeros(1, 2*signal_l);

while true
    shift = input(['Введите размер сдвига (0-', num2str(signal_l), '): ']);

    % Проверка корректности
    if shift < 0
        disp('Сдвиг не может быть отрицательным');
    elseif shift + signal_l > length(new_signal)
```

```

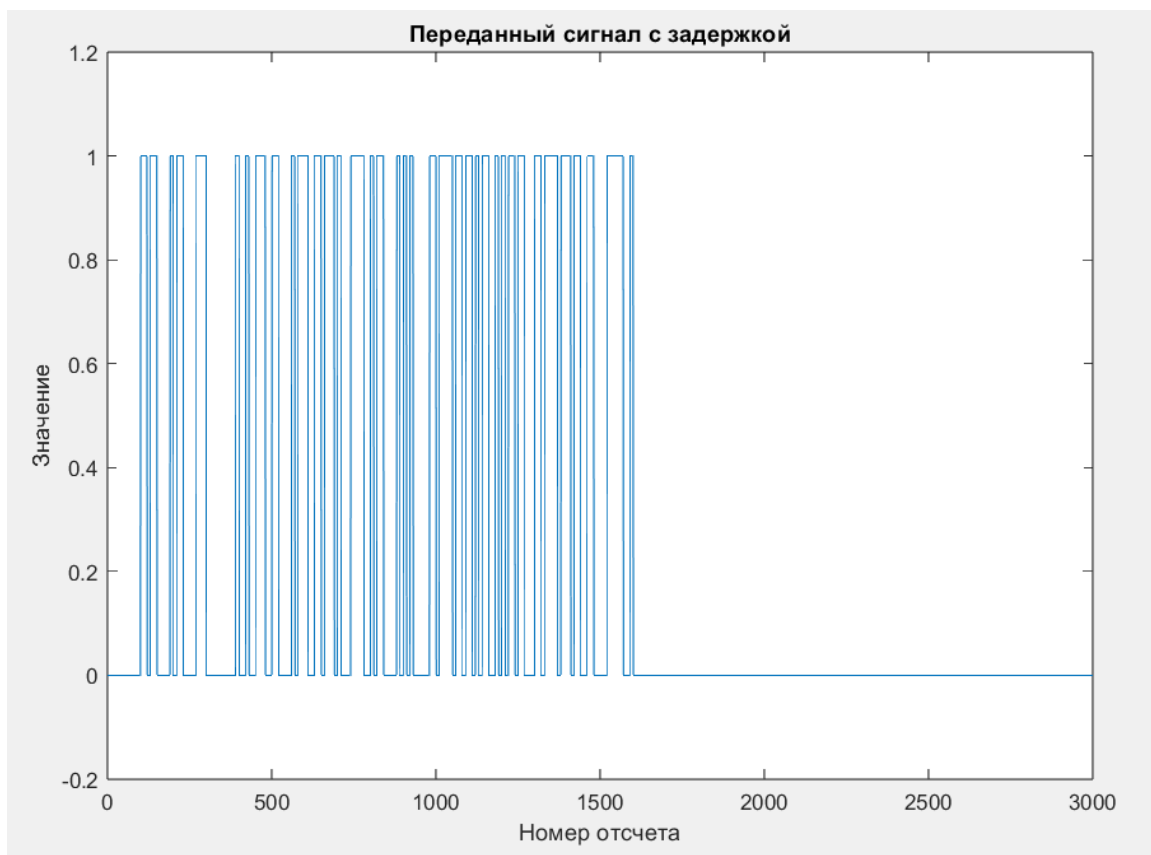
        disp(['Слишком большой сдвиг. Максимум: ', num2str(length(new_signal) -
signal_l)]);
    else
        break;
    end
end

new_signal(shift+1:shift+signal_l) = signal;

figure;
plot(new_signal);
title('Переданный сигнал с задержкой');
ylim([-0.2 1.2]);
xlabel('Номер бита');
ylabel('Значение');

```

Введите размер сдвига (0–1500): 100



Вывод: сигнал размещен в нулевом массиве с заданной задержкой, что моделирует неизвестное время начала приёма пакета у приёмника.

## 7. Добавление шума

В радиоканале всегда присутствуют шумы, которые часто моделируются нормальным (гауссовским) распределением с нулевым математическим ожиданием. Добавление шума позволяет оценить устойчивость системы связи к помехам.

Шум моделируется как белый гауссов шум -  $n \sim N(0, \sigma^2)$

В MATLAB *randn* уже генерирует нормальное распределение  $n \sim N(0,1)$ , чтобы получить  $n \sim N(0, \sigma^2)$  нужно просто умножить на  $\sigma$

Алгоритм:

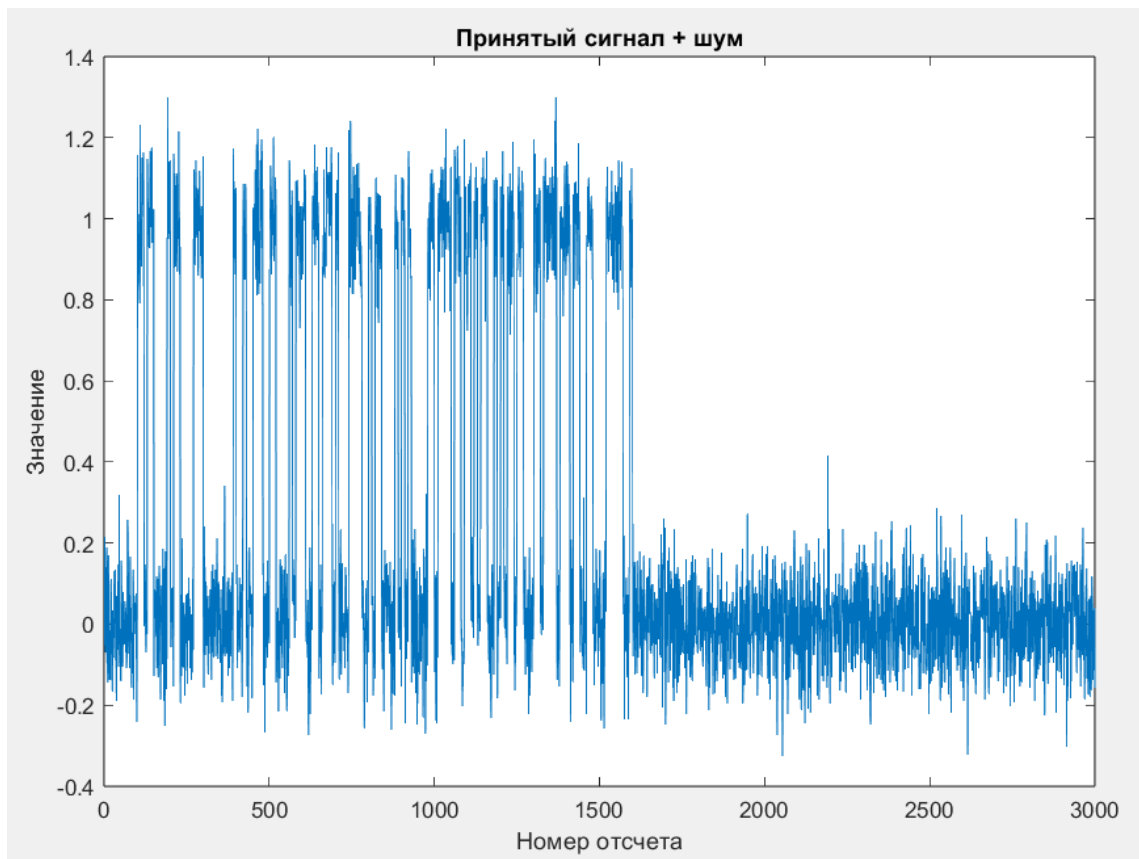
1. Генерируется шум с нормальным распределением и заданной сигмой.
2. Шум поэлементно добавляется к переданному сигналу.

```
sigma = input('Введите сигму: ');
mu = 0;
noise = mu + sigma * randn(1, length(new_signal));

rx = new_signal + noise;

figure;
plot(rx);
title('Принятый сигнал + шум');
xlabel('Номер бита');
ylabel('Значение');
```

```
Введите сигму: 0.1
```



Вывод: к сигналу добавлен белый гауссов шум, моделирующий влияние радиоканала и ухудшение качества приёма.

## 8. Корреляционный прием

Корреляционный приём позволяет обнаружить известную синхропоследовательность в принятом сигнале. Максимум корреляционной функции указывает на начало синхросигнала.

Алгоритм

1. Синхропоследовательность Голда преобразуется во временные отсчеты.
2. Вычисляется нормированная корреляция между принятым сигналом и эталонной последовательностью.
3. По максимуму корреляционной функции определяется начало пакета.
4. Лишние отсчеты отбрасываются.

```
gold_samples = repelem(gold, N);  
corr = norm_corr(rx, gold_samples);
```

```
figure;  
plot(corr);  
xlabel('Номер отсчета');
```

```

ylabel('Значение корреляции');
title('Корреляционная функция');
grid on;

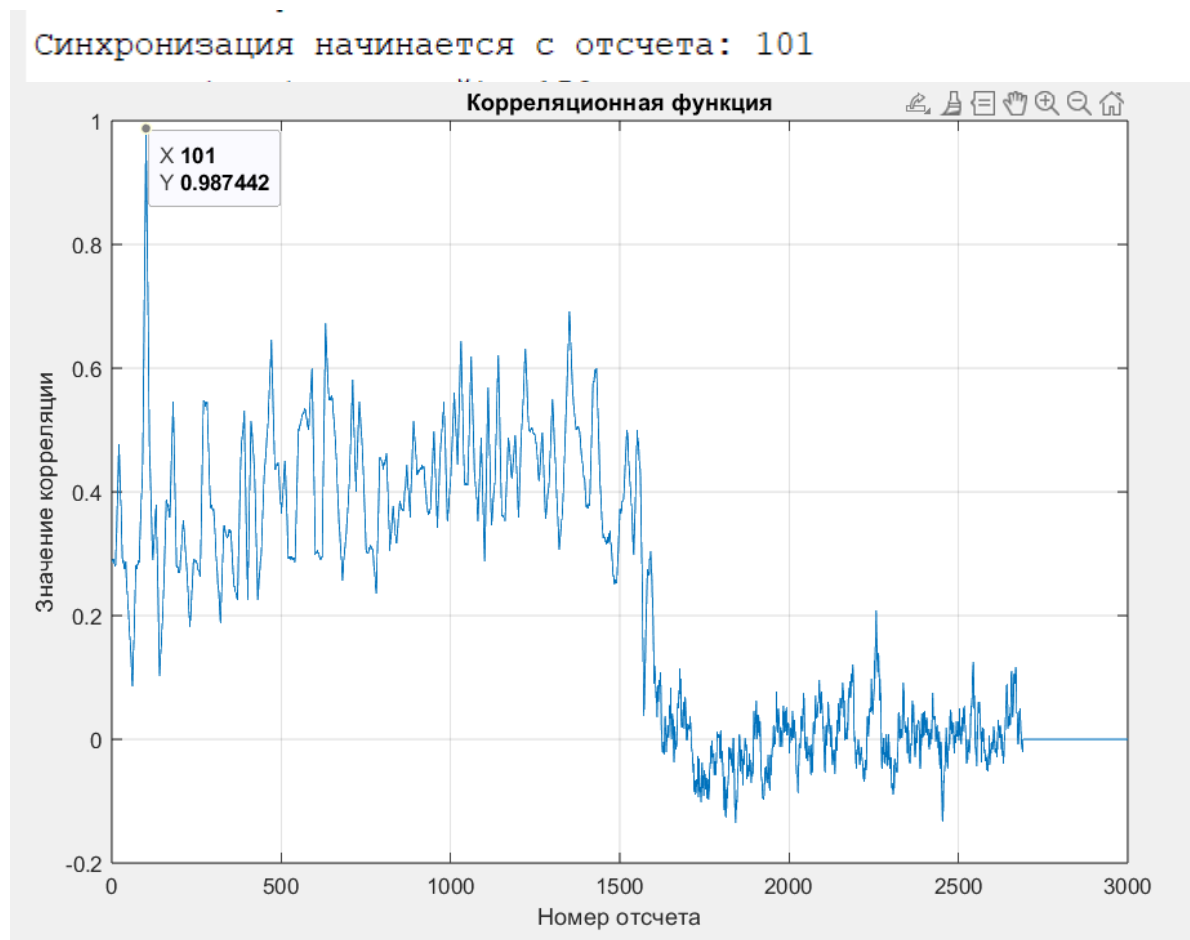
[~, start_sample] = max(corr);

disp(['Синхронизация начинается с отсчета: ', num2str(start_sample)]);

% проверка корректности позиции
if start_sample < 1 || start_sample + length(gold_samples) > length(rx)
    error('Некорректная позиция синхронизации');
end

% удаляем все отсчёты до начала синхросигнала
rx_sync = rx(start_sample : start_sample + length(signal) - 1);

```



Вывод: с помощью корреляционного приёмника определено положение синхросигнала в зашумлённой последовательности, что позволило корректно выделить полезную часть сигнала.

## 9. Принятие решений по отчётам

Для восстановления битовой последовательности необходимо принять решение по каждому символу на основе его временных отсчётов. Используется пороговое решение на основе среднего значения.

Алгоритм:

1. Отсчёты группируются по  $N$  на один бит.
2. Для каждой группы вычисляется среднее значение.
3. Среднее сравнивается с порогом  $P$ .
4. Формируется последовательность принятых бит.

```
P = 0.5;
num_bits = floor(length(rx_sync)/N);
rx_bits = zeros(1, num_bits);

for i = 1:num_bits
    segment = rx_sync((i-1)*N + 1 : i*N);
    rx_bits(i) = mean(segment) > P;
end

disp(['Принято бит (с синхрой): ', num2str(length(rx_bits))]);
```

```
-
Принято бит (с синхрой): 150
```

Вывод: на основе усреднения по  $N$  отсчётам выполнено восстановление битовой последовательности, что позволило снизить влияние шума на процесс детектирования.

## 10. Удаление последовательности синхронизации

Синхропоследовательность используется только для определения начала пакета и не относится к полезным данным.

Алгоритм

1. Из принятой битовой последовательности удаляются первые  $G$  бит.
2. Оставшиеся биты используются для дальнейшей обработки.

```
if length(rx_bits) <= G_l
    error('Not enough bits to remove sync sequence');
end

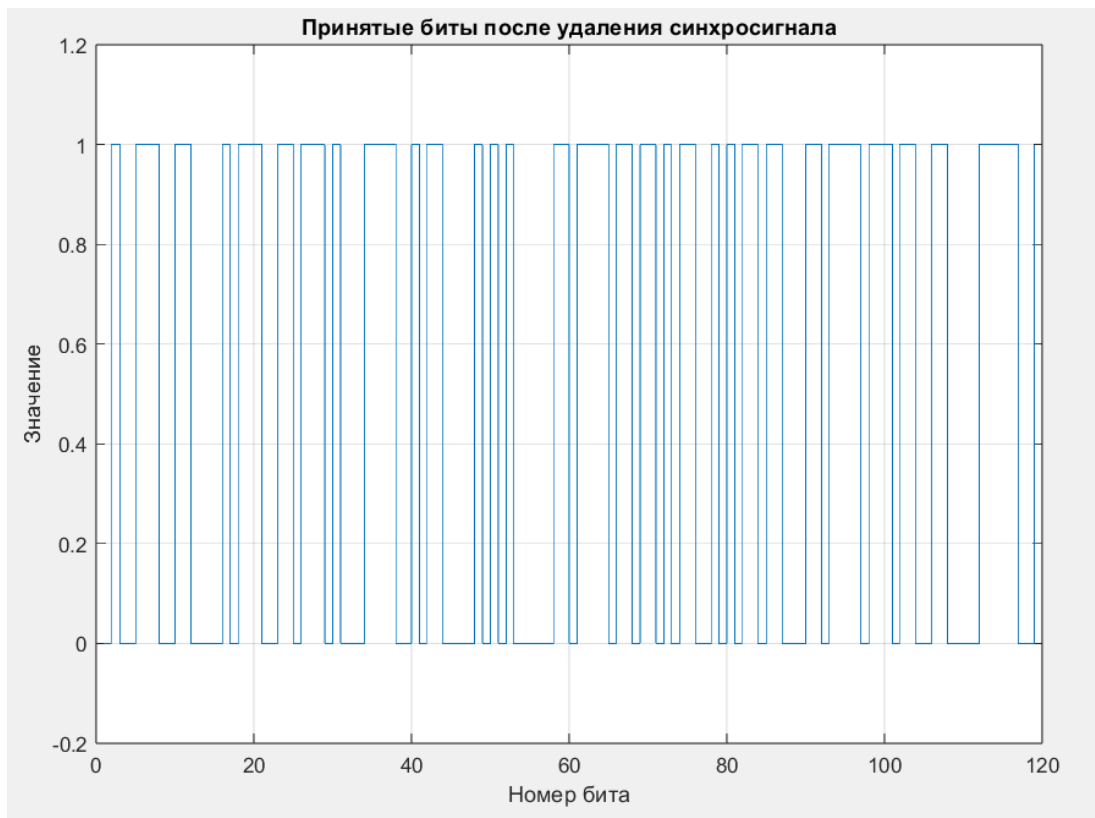
rx_bits = rx_bits(G_l + 1 : end);

disp(['Бит после удаления синхры: ', num2str(length(rx_bits))]);

figure;
stairs(rx_bits);
ylim([-0.2 1.2]);
xlabel('Номер бита');
ylabel('Значение');
title('Принятые биты после удаления синхросигнала');
```

```
grid on;
```

```
Бит после удаления синхры: 119
```



Вывод: синхронизирующая последовательность удалена из принятого массива бит, в результате чего остались только данные и контрольная сумма CRC.

## 11.Проверка CRC

CRC позволяет проверить целостность принятого пакета данных. Если при повторном вычислении CRC остаток равен нулю, считается, что ошибки передачи отсутствуют.

Алгоритм

1. По принятой последовательности повторно вычисляется CRC.
2. Проверяется остаток от деления.
3. В терминал выводится сообщение о наличии или отсутствии ошибок.

```
temp = [rx_bits zeros(1, crc_l)];
```

```
for i = 1:length(rx_bits)-crc_l  
    if temp(i) == 1
```



```

        temp(i:i+crc_l) = xor(temp(i:i+crc_l), G);
    end
end

crc_check = temp(end - crc_l+1:end);

if sum(crc_check) == 0
    disp('CRC correct');
else
    disp('CRC error');
end

```

```

Данные корректны
CRC correct
Полученный текст:

```

Вывод: выполнена проверка корректности приёма данных, которая позволяет определить наличие ошибок без повторной передачи информации.

## 12. ASCII-декодирование

После успешной проверки CRC битовая последовательность может быть преобразована обратно в текст.

Алгоритм:

1. Из битовой последовательности удаляются биты CRC.
2. Биты группируются по 8.
3. Каждая группа переводится в ASCII-код.
4. Формируется и выводится восстановленный текст.

```

if sum(crc_check) == 0
    data_bits = rx_bits(1:end-crc_l);

    disp(['Длина data_bits: ', num2str(length(data_bits))]);
    disp(['Ожидаемая длина: ', num2str(length(bits))]);

    if mod(length(data_bits), 8) ~= 0
        error('Количество битов не кратно 8');
    end

    disp('Первые 32 бита принятых данных:');
    disp(num2str(data_bits(1:min(32, end))));
    disp('Первые 32 бита исходных данных:');
    disp(num2str(bits(1:min(32, end))));

    if length(data_bits) == length(bits)
        bit_errors = sum(data_bits ~= bits);
        disp(['Ошибок в битах: ', num2str(bit_errors)]);
    end

    chars = reshape(data_bits, 8, []);

```

```

decoded = char(bin2dec(num2str(chars)))';

disp('Декодированный текст: ');
disp(decoded);

if strcmp(decoded, name)
    disp('Декодирование успешно!');
else
    disp('Ошибка декодирования!');

    figure;
    subplot(2,1,1);
    stem(bits(1:min(50, end)), 'b', 'filled');
    title('Оригинальные биты');
    ylim([-0.2 1.2]);
    grid on;

    subplot(2,1,2);
    stem(data_bits(1:min(50, end)), 'r', 'filled');
    title('Принятые биты');
    ylim([-0.2 1.2]);
    grid on;
end
else
    disp('Декодирование невозможно из-за ошибки CRC');
end

```

```

Длина принятых данных: 112 бит
Ожидаемая длина: 112 бит
Первые 32 бита принятых данных:
01001110011000010111001101110100
Первые 32 бита исходных данных:
01001110011000010111001101110100
Количество битовых ошибок: 0
Декодированный текст:
NastyaPomelova
Декодирование успешно
>>

```

Вывод: при отсутствии ошибок CRC принятая битовая последовательность успешно декодирована в текст, совпадающий с исходным сообщением.

### 13.Спектры сигналов

Спектральный анализ позволяет оценить частотные характеристики сигналов.

Алгоритм:

1. Вычисляется спектр передаваемого и принимаемого сигналов с помощью БПФ.
2. Анализируется влияние шума на спектр.
3. Изменяется длительность символа ( $N/2$ ,  $N$ ,  $2*N$ ).
4. На одном графике отображаются спектры для разных длительностей СИМВОЛОВ.

```
Fs = 1000;
```

```
% Передаваемый и принимаемый сигнал
```

```
S_tx = abs(fft(signal));
```

```
S_rx = abs(fft(rx));
```

```
f_tx = linspace(0, Fs, length(S_tx));
```

```
f_rx = linspace(0, Fs, length(S_rx));
```

```
figure;
```

```
plot(f_tx, S_tx / max(S_tx), 'b'); hold on;
```

```
plot(f_rx, S_rx / max(S_rx), 'r');
```

```
grid on;
```

```
xlabel('Частота');
```

```
ylabel('Амплитуда');
```

```
title('Спектры передаваемого и принимаемого сигналов');
```

```
ylim([0 0.2]);
```

```
legend('Передаваемый', 'Принимаемый');
```

```
%% Спектры при разной длительности символа
```

```
N_short = N / 2;
```

```
N_long = 2 * N;
```

```
signal_short = repelem(tx_bits, N_short);
```

```
signal_long = repelem(tx_bits, N_long);
```

```
S_short = abs(fft(signal_short));
```

```
S_mid = abs(fft(signal));
```

```
S_long = abs(fft(signal_long));
```

```
f_short = linspace(0, Fs, length(S_short));
```

```
f_mid = linspace(0, Fs, length(S_mid));
```

```
f_long = linspace(0, Fs, length(S_long));
```

```
figure;
```

```
plot(f_short, S_short / max(S_short), 'r'); hold on;
```

```
plot(f_mid, S_mid / max(S_mid), 'g');
```

```
plot(f_long, S_long / max(S_long), 'b');
```

```
grid on;
```

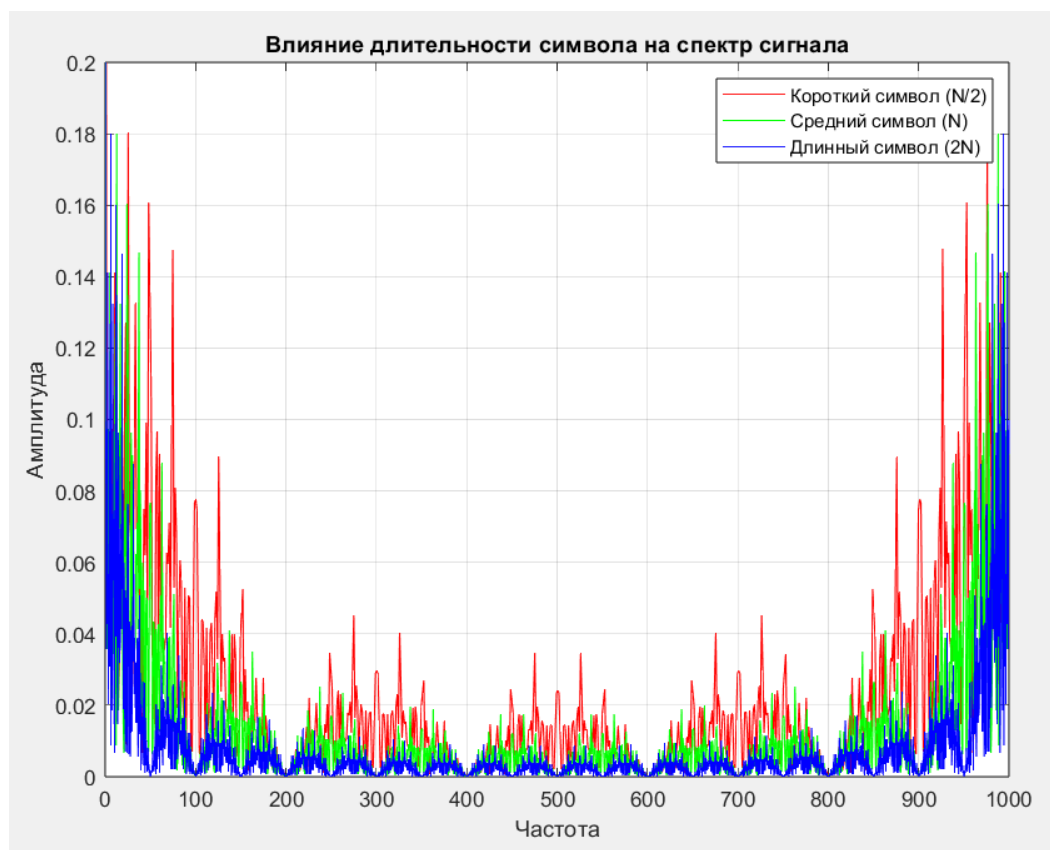
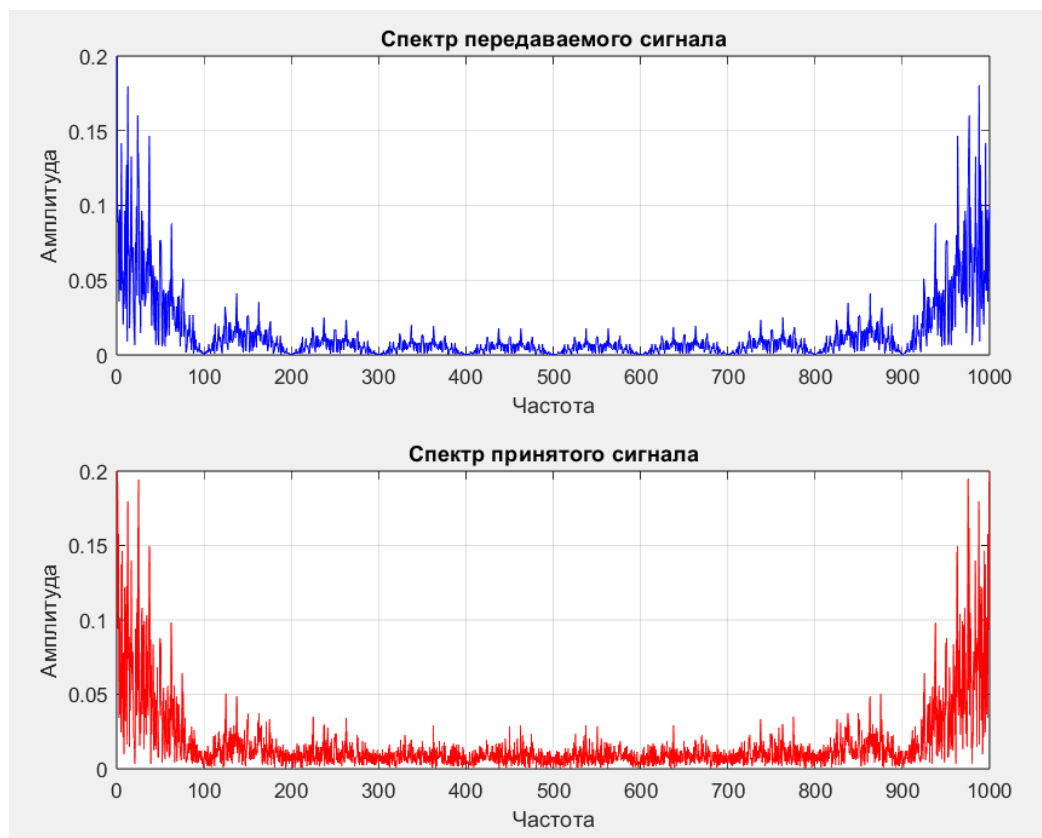
```
xlabel('Частота');
```

```
ylabel('Амплитуда');
```

```
ylim([0 0.2]);
```

```
title('Влияние длительности символа на спектр сигнала');
```

```
legend('Короткий символ (N/2)', 'Средний символ (N)', 'Длинный символ (2N)');
```



Чем короче символ во времени - тем шире спектр по частоте

Вывод: уменьшение длительности символа приводит к расширению спектра сигнала, а увеличение - к его сужению.

### Влияние омеги:

- $\leq 0.4$  – декодирование успешно

```
>> code
Введите имя и фамилию: NastyaPomelova
Битовая последовательность: 0100111001100001011100110111010001111001011000010
CRC: 1111001
Введите размер сдвига (0-1500): 200
Введите сигму: 0.4
Синхронизация начинается с отсчета: 201
Принято бит (с синхрой): 150
Бит после удаления синхры: 119
CRC correct
Длина принятых данных: 112 бит
Ожидаемая длина: 112 бит
Первые 32 бита принятых данных:
01001110011000010111001101110100
Первые 32 бита исходных данных:
01001110011000010111001101110100
Количество битовых ошибок: 0
Декодированный текст:
NastyaPomelova
Декодирование успешно
```

- $> 0.4$  – ошибка декодирования

```
>> code
Введите имя и фамилию: NastyaPomelova
Битовая последовательность: 0100111001100001011100110111010001111001011000010
CRC: 1111001
Введите размер сдвига (0-1500): 200
Введите сигму: 0.5
Синхронизация начинается с отсчета: 201
Принято бит (с синхрой): 150
Бит после удаления синхры: 119
CRC correct
Длина принятых данных: 112 бит
Ожидаемая длина: 112 бит
Первые 32 бита принятых данных:
01001110011000010111001101110100
Первые 32 бита исходных данных:
01001110011000010111001101110100
Количество битовых ошибок: 1
Декодированный текст:
NastyaTomelova
Ошибка декодирования
```

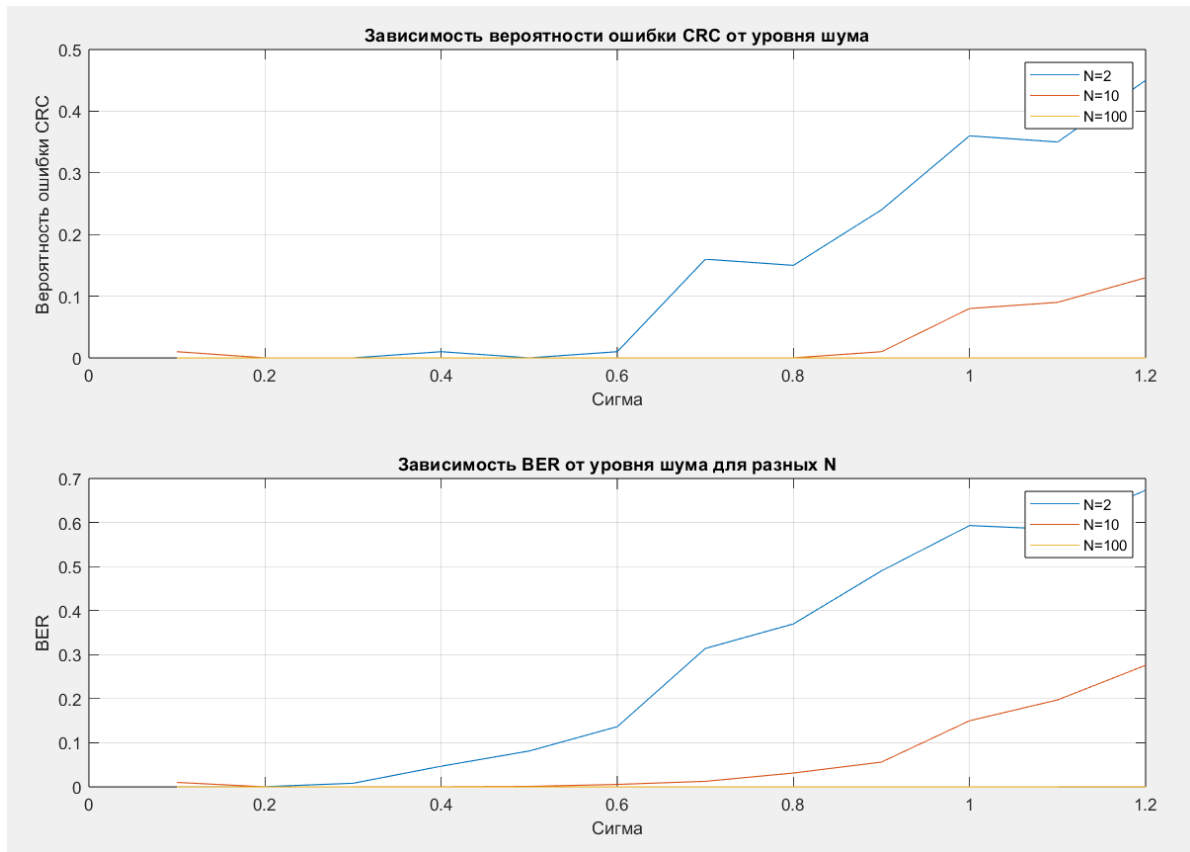
```
>> code
Введите имя и фамилию: NastyaPomelova
Битовая последовательность: 0100111001100001011100110111010001111001011000010
CRC: 1111001
Введите размер сдвига (0-1500): 200
Введите сигму: 1
Синхронизация начинается с отсчета: 201
Принято бит (с синхрой): 150
Бит после удаления синхры: 119
CRC correct
Длина принятых данных: 112 бит
Ожидаемая длина: 112 бит
Первые 32 бита принятых данных:
01000110011000010111001101110100
Первые 32 бита исходных данных:
01001110011000010111001101110100
Количество битовых ошибок: 5
Декодированный текст:
Fast}aPgmelova
Ошибка декодирования
>> code
```

## ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ

Исследовать, как количество отсчетов на бит (N) влияет на устойчивость системы связи к шуму. Анализируются показатели:

1. BER (Bit Error Rate) - вероятность ошибки на бит

## 2. Вероятность ошибки CRC - вероятность того, что CRC обнаружит ошибку



```
sigmas = 0.1:0.1:1.2;
Samples = [2 10 100];
nums = 100;

crc_errors_val = zeros(length(Samples), length(sigmas));
ber_val = zeros(length(Samples), length(sigmas));

tx_bits_full = tx_bits; %[Gold + bits + CRC]

for idx = 1:length(Samples)
    N = Samples(idx);
    fprintf('Обработка N = %d\n', N);

    for ydx = 1:length(sigmas)
        sigma = sigmas(ydx);
        crc_er = 0;
        bit_er = 0;
        total_bits = 0;

        for zdx = 1:nums
            signal = repelem(tx_bits_full, N);
            signal_l = length(signal);

            max_shift = signal_l;
            shift = randi([0, max_shift]); %случайный сдвиг
            new_signal = zeros(1, 2*signal_l);
            new_signal(shift+1 : shift+signal_l) = signal;

            noise = sigma * randn(size(new_signal));
            rx = new_signal + noise;
```

```

gold_samples = repelem(gold, N);
corr = norm_corr(rx, gold_samples);
[~, start_sample] = max(corr);

%проверка корректности позиции синхронизации
if start_sample < 1 || start_sample + signal_l > length(rx)
    crc_er = crc_er + 1;
    bit_er = bit_er + length(bits);
    total_bits = total_bits + length(bits);
    continue;
end

%убираем лишнее до начала сигнала
rx_sync = rx(start_sample : start_sample + signal_l - 1);

%демодуляция
num_bits_total = length(tx_bits_full);
rx_bits_total = zeros(1, num_bits_total);
for i = 1:num_bits_total
    segment = rx_sync((i-1)*N + 1 : i*N);
    rx_bits_total(i) = mean(segment) > P;
end

%убираем голда
rx_bits = rx_bits_total(G_l + 1 : end);

%вычисление BER
if length(rx_bits) >= length(bits) + crc_l
    %извлекаем только данные
    data_rx = rx_bits(1:length(bits));
    %сравниваем принятые данные с исходными, считаем ошибки
    bit_er = bit_er + sum(data_rx ~= bits);
    %счетчик кол-ва ошибок
    total_bits = total_bits + length(bits);
else %если принято недостаточно битов - все ошибочные
    bit_er = bit_er + length(bits);
    total_bits = total_bits + length(bits);
end

%проверка CRC
temp = [rx_bits zeros(1, crc_l)];
for i = 1:length(rx_bits) - crc_l
    if temp(i) == 1
        temp(i:i+crc_l) = xor(temp(i:i+crc_l), G);
    end
end
crc_check = temp(end - crc_l + 1 : end);
if sum(crc_check) ~= 0
    crc_er = crc_er + 1;
end

%вероятность ошибки CRC = количество ошибок / общее количество испытаний
crc_errors_val(idx, ydx) = crc_er / nums;
%BER = количество ошибочных битов / общее количество переданных битов
ber_val(idx, ydx) = bit_er / total_bits;
end
end

figure;
subplot(2,1,1);
plot(sigmas, crc_errors_val');

```



```
xlabel('Сигма');
ylabel('Вероятность ошибки CRC');
legend('N=2', 'N=10', 'N=100');
grid on;
title('Зависимость вероятности ошибки CRC от уровня шума');

subplot(2,1,2);
plot(sigmas, ber_val);
xlabel('Сигма');
ylabel('BER');
legend('N=2', 'N=10', 'N=100');
grid on;
title('Зависимость BER от уровня шума для разных N');
```

## ВЫВОДЫ

В ходе выполнения расчетно-графической работы мною была смоделирована цифровая система связи, включающая этапы кодирования, синхронизации, передачи по зашумлённому каналу и приёма сигнала. Реализованы алгоритмы корреляционного приёма, принятия решений и контроля ошибок с помощью CRC. Проведён спектральный анализ сигналов и показано влияние длительности символа на ширину спектра. Полученные результаты подтверждают корректность работы модели и соответствуют теоретическим представлениям о цифровых системах связи.

## КОД

Ссылка на github

